Motor Calibration and Automatic PID Tuning for a Four-Wheel Skid-Steer Robot

Theory, Firmware (`PID_TEST.cpp`), Host Dashboard (`motor_calibrate.py`), and Operating Procedure

[Author name]
[Institution / Department]

2026-10-17

Abstract

This report documents a complete workflow for identifying, tuning and verifying the wheel-speed controllers of a four-wheel, encoder-equipped, DC-motor skid-steer robot built around an ESP32-S3. The workflow has two halves. The first is a dedicated firmware image, `PID_TEST.cpp`, which runs four independent speed loops (feedforward plus proportional-integral action, with separate forward and reverse gain sets) and exposes a plain-text serial protocol. The second is a Python dashboard, `motor_calibrate.py`, which drives that protocol: it records open-loop step responses, fits a first-order-plus-dead-time (FOPDT) model with a motor dead-band for every wheel and direction, derives PI gains with the SIMC rules, verifies and detunes them with closed-loop steps, adds a feedforward term, sweeps the speed range, and finally exports the gains as JSON and as a C++ header consumed by the production robot firmware. The report gives the underlying control theory, a line-by-line description of both programs, the exact operating procedure, safety rules, a troubleshooting guide, a list of known limitations, and empty result tables and photo slots to be filled in during commissioning.

Contents

1 Introduction **4**

1.1 Motivation . . . 4

1.2 Scope . . . 4

1.3 Notation and conventions . . . 4

1.4 Safety . . . 5

2 System Description **6**

2.1 Hardware . . . 6

2.2 Wheel naming and index order . . . 7

2.3 Motor driver interface . . . 7

2.4 Speed measurement . . . 7

2.5 System architecture . . . 8

3 Control Theory **9**

3.1 Plant model: first order plus dead time with dead-band . . . 9

3.2 Identification from open-loop steps . . . 9

3.3 PI control law . . . 10

3.3.1 The feedforward term . . . 10

3.3.2 Separate forward and reverse gain sets . . . 10

3.3.3 Integrator limit and anti-windup . . . 11

3.3.4 Derivative action . . . 11

3.4 PI tuning: the SIMC rules . . . 11

3.4.1 Choice of τ_c and the shared value . . . 11

3.5 Closed-loop verification and detuning . . . 11

3.6 Feedforward speed-up . . . 12

3.7 Speed sweep . . . 12

4 Test Firmware **13**

4.1 Purpose . . . 13

4.2 Control loop timing . . . 13
4.3 The `SimplePID` class . . . 14
4.4 Serial command protocol . . . 14
4.5 Telemetry . . . 15
4.6 Initial values . . . 15

5 Host Software **16**
5.1 Dependencies and configuration . . . 16
5.2 Tunable constants in the source . . . 16
5.3 Program structure . . . 17
5.3.1 Per-motor memory . . . 17
5.3.2 Preventing feedback loops in the GUI . . . 17
5.3.3 Capture mechanism . . . 18
5.4 Graphical interface . . . 18

6 Numerical Routines **20**
6.1 Role of the module . . . 20
6.2 Helper functions . . . 20
6.3 Gain and dead-band by regression . . . 21
6.4 Time constant and dead time by grid search . . . 21
6.5 The shared closed-loop time constant . . . 22
6.6 SIMC gains . . . 22
6.7 Step metrics . . . 22
6.8 Worked example on a simulated motor . . . 22
6.8.1 Identification . . . 23
6.8.2 Gains and verification . . . 23
6.8.3 Effect of τ_c and of feedforward . . . 24

7 The Autotune Algorithm Step by Step **26**
7.1 Overview . . . 26
7.2 Pass 1: identification . . . 26
7.3 Pass 2: computing gains . . . 27
7.4 Pass 3: verification . . . 27
7.5 Saving . . . 27
7.6 Estimated duration . . . 28

8 Other Functions of the Dashboard **29**
8.1 Step test . . . 29

8.2 Speed sweep . . . 29
8.3 Flashing from the dashboard . . . 29

9 Operating Procedure **30**
9.1 One-time set-up . . . 30
9.2 Direction check (before any tuning) . . . 30
9.3 Tuning session . . . 31
9.4 Recording the results . . . 32
9.5 Tuning on the floor versus on the stand . . . 33
9.6 When to retune . . . 33

10 Troubleshooting **34**

11 Limitations **36**
11.1 Review of `autotune_math.py` . . . 36
11.2 Observations on the supplied source . . . 37
11.3 Possible extensions . . . 37

A Quick Reference **38**
A.1 Command cheat sheet . . . 38
A.2 Procedure on one page . . . 38

B Fill-in Photo Checklist **39**

List of Figures

1.1 Robot on a stand with all four wheels lifted for the autotune. 5

2.1 Close-up of one motor, encoder and driver wiring. 6

2.2 Tuning set-up. The dashboard commands the test firmware over USB serial and reads back the four wheel speeds; the result is written to files that the production firmware includes at build time. 8

3.1 Open-loop step response of the model (3.2). θ is the delay before the speed starts to rise; after $\theta + \tau$ the speed has reached 63 % of its final value. 9

5.1 Screenshot of the dashboard during a step test. 18

6.1 Open-loop step at 180 PWM on the simulated motor and the fitted model ($\hat{\tau} = 0.183\,\mathrm{s}$, $\hat{L} = 0.0875\,\mathrm{s}$). The steps in the data are single encoder counts (1.6 rpm). Simulation, not measured data. 23

6.2 Closed-loop 30 rpm step on the simulated motor. Round 1 overshoots, the detuned round 2 is accepted, and adding feedforward to a plain PI loop makes the overshoot worse. Simulation, not measured data. 24

7.1 Flow of `autotune_worker`. Any step can be aborted with STOP MOTORS. . . . 26

9.1 Direction check: arrow painted on each wheel showing the forward direction. . . . 31

9.2 Plot of the four wheel speeds during STEP TEST after autotune. 32

List of Tables

1.1 Symbols used in this report. 5

2.1 Hardware summary. Values marked **[TODO:]** must be completed by the builder. 6

4.1 Serial commands accepted by `PID_TEST.cpp`. Example: `TA30` sets all wheels to 30 rpm; `P25.5` sets K_p of wheel 2 (FR) to 5.5 — note that the value starts right after the selector. 15

5.1 Environment variables read by the dashboard. 16

5.2 Autotune constants in `motor_calibrate.py`. 17

5.3 Dashboard controls. 19

6.1 Public functions of `autotune_math.py`. 20

6.2 True and identified parameters of the simulated motor. 23

6.3 `verify_motor` on the simulated motor, 30 rpm step. A round passes if overshoot $\leq 5\,\%$ and ripple ≤ 3 rpm. 23

6.4 Effect of the design parameter τ_c (simulated motor, $F = 0$, 30 rpm step). Values of τ_c below L give the same gains, because of the $\max(\tau_c, L)$ floor. 24

6.5 Effect of the feedforward fraction f ($F = f/K$) with the round-1 gains and no detuning (simulated motor). 24

9.1 Identified plant parameters (copy from the console). **[TODO: Fill in.]** 32

9.2 Final gains. **[TODO: Fill in from `tuned_gains.json`.]** 32

9.3 Closed-loop verification at 30 rpm. **[TODO: Fill in.]** 33

Chapter 1

Introduction

1.1 Motivation

A mobile robot that navigates autonomously (for example with a ROS 2 stack) receives velocity commands in physical units: metres per second and radians per second. Somewhere below the navigation layer these commands must become voltages on four motors. The quality of that conversion limits everything above it. If the four wheels do not track their speed targets equally well, the robot curves when told to drive straight, turns by an amount different from the commanded rate, and produces wheel odometry that disagrees with the true motion.

Hand-tuning four motors, each in two directions, is slow and not repeatable. Every motor has its own dead-band, gain and time constant, and the forward and reverse characteristics of a brushed motor behind a gearbox are generally not identical. The purpose of this work is to replace manual tuning with a measured, documented, repeatable procedure.

1.2 Scope

This document covers:

- the test firmware `PID_TEST.cpp`, which is flashed to the ESP32-S3 while tuning;
- the host program `motor_calibrate.py`, a Tk/Matplotlib dashboard that tunes through the serial port;
- the numerical routines it imports from `autotune_math.py`, documented from the source in Chapter 6 and exercised on a simulated motor;
- the hand-off of the tuned gains to the production robot firmware through `gains.h`.

It does not cover the ROS 2 layers (micro-ROS, EKF, SLAM), except where they consume the tuned gains.

1.3 Notation and conventions

Table 1.1 lists the symbols used throughout. Rotational speeds are in revolutions per minute (rpm) at the *wheel*, i.e. after the gearbox. The PWM command is an integer in $[-255, 255]$ and is dimensionless in the sense of an 8-bit duty value. The sample period is $T_s = 0.05\,\mathrm{s}$.

Table 1.1: Symbols used in this report.

Symbol	Unit	Meaning
u	PWM counts	Motor command, $-255 \dots 255$
u_0 (`pwm0`)	PWM counts	Dead-band: smallest PWM that produces steady motion
y, ω	rpm	Measured wheel speed
r	rpm	Speed reference (target)
K	rpm / PWM	Static (steady-state) gain of the plant
τ	s	Dominant time constant of the plant
θ, L	s	Effective dead time of the plant
τ_c	s	Desired closed-loop time constant
K_p	PWM / rpm	Proportional gain
K_i	PWM / (rpm·s)	Integral gain (rate form, $K_i = K_p/\tau_I$)
K_d	PWM·s / rpm	Derivative gain (not used by the autotune)
F (`ff_gain`)	PWM / rpm	Feedforward gain; ideal value is $1/K$
B (`ff_offset`)	PWM counts	Feedforward offset, equal to u_0 after autotune
N	counts / rev	Encoder counts per wheel revolution (748)
T_s	s	Sample period (0.05)

1.4 Safety

Warning. The autotune spins every motor up to the highest test PWM (210 counts in the default configuration, and up to 250 counts for the refinement steps) with no speed limit other than the PWM itself. **Lift the robot so that all four wheels are off the ground and cannot touch anything before pressing AUTOTUNE, STEP TEST or SPEED SWEEP.** Reverse runs are included. Keep hands, cables and loose objects away from the wheels. The *STOP MOTORS* button sends `TA0` and aborts a running autotune, but it cannot help if the serial link is down. Keep a hand on the power switch.

PHOTO PLACEHOLDER

Robot on a stand with all four wheels lifted for the autotune.

Figure 1.1: Robot on a stand with all four wheels lifted for the autotune.

Chapter 2

System Description

2.1 Hardware

The platform is a four-wheel skid-steer chassis. Each wheel is driven by a brushed DC gear-motor with a quadrature encoder, each through one channel of an H-bridge driver, all controlled by an ESP32-S3. Figures and values specific to the build are to be filled in below.

Table 2.1: Hardware summary. Values marked **[TODO:]** must be completed by the builder.

Item	Value
Microcontroller	ESP32-S3 (Arduino framework, PlatformIO)
Motors	**[TODO: motor model, rated voltage, gear ratio]**
Motor driver	**[TODO: driver model]**; one PWM input (`IN1`) and one direction input (`IN2`) pe
Encoder	Quadrature, read in half-quadrature mode, $N = 748$ counts per wheel revolution
Wheel diameter	76.2 mm (radius 0.0381 m, circumference 0.2394 m)
Battery	**[TODO: chemistry, cell count, nominal voltage]**
PWM frequency	20 kHz (`analogWriteFrequency(20000)`)
Serial link (test firmware)	115200 baud, 8N1

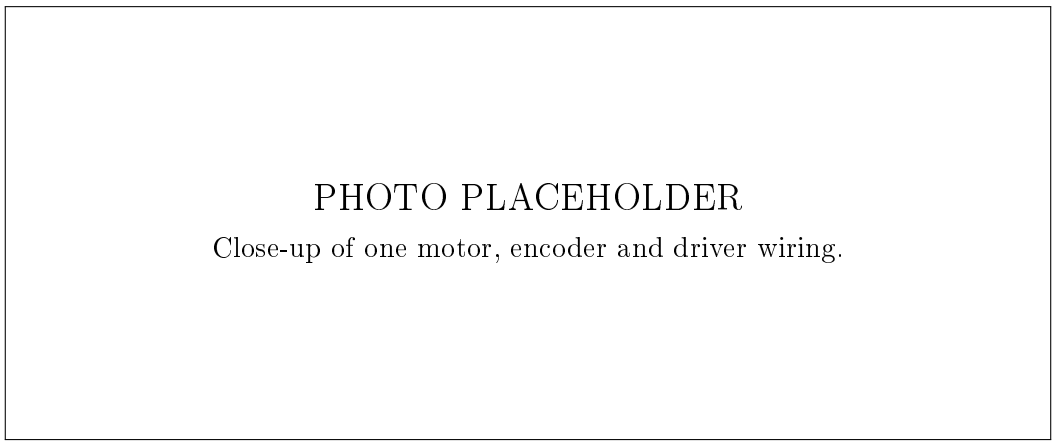


Figure 2.1: Close-up of one motor, encoder and driver wiring.

2.2 Wheel naming and index order

Everywhere in `PID_TEST.cpp` and in the dashboard the wheels are indexed in the order **front-left, front-right, rear-left, rear-right**, and are addressed on the serial port by a selector character:

Array index	Selector	Name	Dashboard colour
0	`1`	FL	red
1	`2`	FR	green
2	`3`	RL	blue
3	`4`	RR	orange

The selector `A` addresses all four wheels. Left and right are from the robot's own point of view, standing behind it and looking in the driving direction. The production firmware (`main.cpp`) stores its wheels in a different array order (FL, RL, FR, RR), but the generated header `gains.h` uses named constants (`GAINS_FL`, `GAINS_FR`, `GAINS_RL`, `GAINS_RR`), so the difference is harmless.

2.3 Motor driver interface

Each motor uses two pins. `IN2` selects the direction and `IN1` carries the PWM. The function `set_raw_motor(in1, in2, pwm)` implements:

- $u = 0$: `IN2` low, `IN1` duty 0 (coast or brake, depending on the driver);
- $u > 0$: `IN2` low, `IN1` duty u;
- $u < 0$: `IN2` high, `IN1` duty $255 + u = 255 - |u|$.

In the reverse case the PWM is *inverted*: with `IN2` held high the driver sees a high level for most of the period, so the effective drive is $|u|$. This is consistent with a driver in which the two inputs select the output polarity (check the driver datasheet for the exact truth table). The sign of motion for a given sign of u depends on the wiring, which is why a per-wheel constant `MOTOR_DIR_*` (value ± 1, defined in `pins.h`) multiplies the command and the encoder reading. With this convention a positive speed always means "robot rolls forward" regardless of how each motor was wired.

2.4 Speed measurement

The speed is computed every control period from the change of the encoder count Δc:

$$\omega[k] = \frac{\Delta c}{N} \cdot \frac{60}{T_s} \cdot d, \qquad d = \texttt{MOTOR_DIR} \in \{-1, +1\}. \tag{2.1}$$

The count is read with `ESP32Encoder::getCount()` in half-quadrature mode (`attachHalfQuad`), and the internal weak pull-ups are enabled.

Quantisation

One encoder count corresponds to a speed step of

$$\Delta\omega_{\min} = \frac{60}{N\,T_s} = \frac{60}{748 \times 0.05} \approx 1.60\,\text{rpm}. \tag{2.2}$$

This is a large resolution compared with the speeds used for verification (≈ 30 rpm, about 19 counts per sample). It has three consequences used later: (i) a steady-state ripple of one or two counts is quantisation hunting, not a control problem; (ii) any derivative action amplifies this step; and (iii) the ripple limit of the autotune must be set above this value.

2.5 System architecture

Figure 2.2 shows the data flow between the host and the microcontroller during tuning.

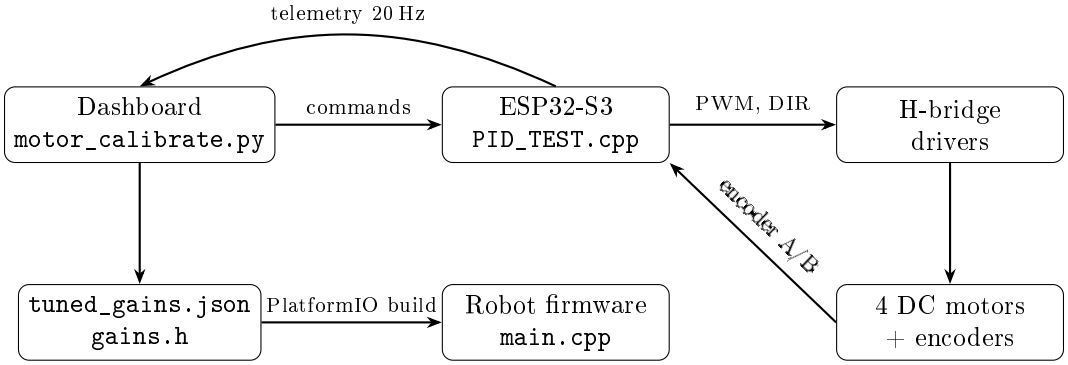


Figure 2.2: Tuning set-up. The dashboard commands the test firmware over USB serial and reads back the four wheel speeds; the result is written to files that the production firmware includes at build time.

Chapter 3

Control Theory

3.1 Plant model: first order plus dead time with dead-band

A permanent-magnet DC motor driven through a gearbox, with its inertial load, behaves for small variations around an operating point like a first-order lag. Static friction and the minimum voltage needed to overcome it add a dead-band, and sampling, driver and mechanical backlash add a short delay. The model used by the autotune is therefore

$$G(s) = \frac{K\, e^{-\theta s}}{\tau s + 1}, \qquad \omega_{ss} = K\,(u - u_0) \;\text{ for } u > u_0, \;\; \omega_{ss} = 0 \text{ otherwise.} \tag{3.1}$$

For an open-loop PWM step of height u applied at $t = 0$ to a wheel at rest, the response is

$$\omega(t) = K\,(u - u_0)\left(1 - e^{-(t-\theta)/\tau}\right), \qquad t \geq \theta, \tag{3.2}$$

and zero before θ. Three parameters, K, τ and θ, plus the dead-band u_0, completely describe one motor in one direction. The model is linear only above the dead-band, which is why the identification uses several PWM levels and a straight-line fit of steady-state speed against PWM; the intercept of that line on the PWM axis is u_0.

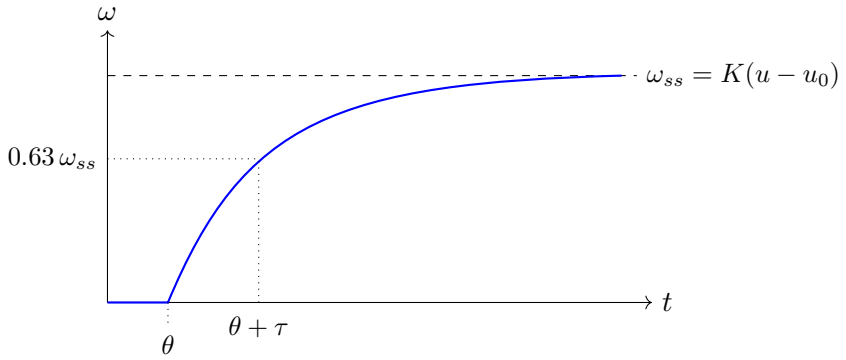


Figure 3.1: Open-loop step response of the model (3.2). θ is the delay before the speed starts to rise; after $\theta + \tau$ the speed has reached 63 % of its final value.

3.2 Identification from open-loop steps

For each wheel and each direction the autotune applies a PWM step from rest, records the speed at T_s, fits a straight line to the steady-state speeds (Section 6.3) and fits τ and θ to each recorded

trace (Section 6.4). This is done at several PWM levels. In the default configuration three levels are used first (150, 180, 210 counts), and then, when `ADAPTIVE_LEVELS` is enabled, further levels are placed where the fitted model predicts the wheel will reach the speeds in `REFINE_RPMS`:

$$u_{\text{refine}} = \text{round}(\min\{250,\ \max\{40,\ u_0 + v_{\text{target}}/K\}\})\,. \tag{3.3}$$

A refine level that falls within 8 counts of an existing level is skipped. The reason for the second stage is that the three initial levels all produce high speeds, so a model fitted only to them says little about the low-speed region where the dead-band and stiction dominate. After the refine steps the model is fitted again using all levels.

The program prints, per wheel and direction, the quantities `K`, `pwm0`, `tau`, `L`, the RMS fit error, a linearity error and the list of PWM levels used. *Reading these lines is the main diagnostic of the whole process*: a large RMS fit error means the first-order model is a poor description (for example because of a slipping belt, a loose coupling or a battery that sags during the step); a large linearity error means the speed is not proportional to $(u - u_0)$ over the tested range.

3.3 PI control law

The controller used in both firmware images has the structure

$$u[k] = \underbrace{F\,r[k] + \text{sgn}(r[k])\,B}_{\text{feedforward}} + \underbrace{K_p\,e[k] + I[k]}_{\text{feedback}}, \qquad e = r - \omega, \tag{3.4}$$

$$I[k] = I[k-1] + K_i\,T_s\,e[k]. \tag{3.5}$$

The output is clamped to $[-255, 255]$. A target of exactly zero forces $u = 0$ and clears the integrator and the previous error, so a stopped wheel carries no wind-up into the next move. When the sign of the target changes, the integrator is also cleared, because an integrator filled for forward motion would act against the new direction.

3.3.1 The feedforward term

In steady state a wheel at speed r needs $u = u_0 + r/K$. The feedforward term supplies exactly this when $F = 1/K$ and $B = u_0$. Two benefits follow. First, the integrator no longer has to build up the whole drive level, so the rise is faster and the overshoot smaller. Second, the dead-band is bridged immediately, which matters at low speed. A feedforward that is too large causes overshoot, because the loop then starts above the target; the autotune therefore starts from $F = 0$ (offset only, `FF_SCALE = 0`) and raises it afterwards only as far as the verification limits allow (Section 3.6).

3.3.2 Separate forward and reverse gain sets

A brushed motor with a gearbox rarely behaves identically in the two directions: brush position, gear mesh and the driver's inverted PWM in reverse all differ. The firmware therefore holds two complete sets $\{K_p, K_i, K_d, F, B\}$ per wheel. The set is chosen by the *sign of the target*, not of the measured speed. In the serial protocol the forward set uses upper-case letters (`P I D F B`) and the reverse set the corresponding lower-case letters (`p i d f b`).

3.3.3 Integrator limit and anti-windup

In `PID_TEST.cpp` the integrator state (in rpm·s) is clamped to $\pm 255/K_i$, so that the integral contribution $K_i I$ can never exceed the full PWM range. (A comment in the source records that a fixed limit of 150 had allowed 750 PWM at $K_i = 5$.) With $K_i = 0$ the integrator is held at zero. The production firmware uses a different but compatible scheme, stopping integration while the output is saturated and the error pushes further into saturation (conditional integration). Both prevent the long recovery tail that a wound-up integrator causes.

3.3.4 Derivative action

The test firmware supports K_d, but the autotune always sets it to zero, and the dashboard slider is limited to 0–2. The reason is the quantisation in Equation (2.2): a single count of noise in one sample produces a change in the computed derivative of $\Delta\omega_{\min}/T_s \approx 32\,\text{rpm/s}$, multiplied by K_d. On a wheel whose steady-state ripple is already one count, derivative action turns that ripple into audible motor chatter. Derivative action is only worth trying with a filtered speed estimate, which this firmware does not have.

3.4 PI tuning: the SIMC rules

The gains are computed by the Skogestad IMC (SIMC) rules for a first-order-plus-dead-time process under PI control. For the plant (3.1) with desired closed-loop time constant τ_c:

$$K_p = \frac{1}{K} \cdot \frac{\tau}{\tau_c + \theta}, \qquad \tau_I = \min\{\tau,\ 4(\tau_c + \theta)\}, \qquad K_i = \frac{K_p}{\tau_I}. \tag{3.6}$$

Equation (3.6) is the standard published form and is what `simc_pi` computes (Section 6.6), followed by caps of 20 on K_p and 60 on K_i. Because K has units of rpm/PWM, K_p has units of PWM/rpm, matching the firmware. The closed-loop response to a setpoint step is approximately a first-order lag with time constant τ_c plus the delay θ. The single design parameter τ_c trades speed against robustness: decreasing it gives a faster response but a smaller gain and phase margin; the rule of thumb is $\tau_c \geq \theta$ (the "tight control" limit).

3.4.1 Choice of τ_c and the shared value

The dashboard sets τ_c once, as the largest of the lower bounds over every wheel and direction m (`common_tau_c`, Section 6.5):

$$\tau_c = \max_m \max\{\,\texttt{TAU_C_FACTOR}\cdot\tau_m,\ \texttt{TAU_C_L_MULT}\cdot\theta_m\,\}, \qquad \texttt{TAU_C_FACTOR} = 0.25,\ \texttt{TAU_C_L_MULT} = 1.25. \tag{3.7}$$

The value is common to all motors and both directions, so the slowest or most delayed plant sets it. The point of a common value is that the four wheels then respond with the same closed-loop speed, even if their plants differ. Different closed-loop speeds across the wheels would produce a transient yaw error whenever the robot accelerates, even if every wheel reached the same final speed.

3.5 Closed-loop verification and detuning

Model-based gains are only as good as the model, so each tuned pair is tested with a closed-loop step. The dashboard measures:

Overshoot the maximum of the 3-point-smoothed speed above the target, as a percentage of the target;

Rise time (90 %) the first time the smoothed speed reaches 90 % of the target (linearly interpolated between samples);

Settling time the end of the last sample of the smoothed speed that lies outside a band of max{5 % of the target, 1.3 rpm} around the target (0 if it never leaves the band);

Ripple the peak-to-peak variation of the raw (unsmoothed) speed over the last 30 % of the window;

Final value the mean raw speed over the same last 30 %.

A step passes when overshoot is at most `MAX_OVERSHOOT_PCT` (5 %) and ripple is at most `RIPPLE_MAX_RPM` (3 rpm). If it fails, K_p and K_i are both multiplied by `DETUNE_FACTOR` (0.75), the step is repeated, and this can occur at most `MAX_DETUNE_ROUNDS` (3) times. Scaling both gains by the same factor leaves the integral time $\tau_I = K_p/K_i$ unchanged and lowers only the loop bandwidth, which is the correct direction for reducing overshoot.

The verification target is `VERIFY_RPM = 30`, reduced to 80 % of the largest steady-state speed reached in the identification if the wheel cannot reach 30 rpm, with a floor of 10 rpm.

3.6 Feedforward speed-up

After the PI settings pass (and only if the rise time was measurable and the overshoot within the limit), the autotune tries larger feedforward gains. It sets $F = f/K$ in turn for $f \in \{0.4, 0.7, 1.0\}$ (`FF_SPEEDUP`) and repeats the verification step each time. A setting is kept only if it still passes the limits *and* shortens the 90 % rise time by more than 20 ms; the first setting that fails to improve ends the search, and the best setting so far is retained. The result is the fastest feedforward that does not cost overshoot or ripple.

3.7 Speed sweep

A linear model identified near the middle of the speed range does not guarantee uniform behaviour at the ends. The speed sweep commands all four motors together to each speed in `SWEEP_SPEEDS` (10, 20, 30, 50, 70, 100, 120 rpm), forward and then reverse, with the *current* gains. For each speed it reports the final speed of each wheel, the spread between the wheels and the largest miss from the target, and flags the speed if either exceeds `SWEEP_SPREAD_RPM` (2.5 rpm; this is less than two encoder counts, see Equation (2.2)). A flagged speed at the high end usually means the PWM has saturated: the largest speed a wheel can reach is approximately

$$\omega_{\max} \approx K\,(u_{\max} - u_0), \qquad u_{\max} = 250 \text{ (production firmware) or } 255 \text{ (test firmware).} \tag{3.8}$$

Compare this with the identified K and `pwm0` before concluding that a flag at 100 or 120 rpm is a tuning problem.

Chapter 4

Test Firmware: PID_TEST.cpp

4.1 Purpose

`PID_TEST.cpp` is a minimal firmware whose only job is to run four speed loops with runtime-adjustable gains and report the measured speeds. It deliberately contains none of the ROS code. It is flashed using the `pid_test` PlatformIO environment (the name is configurable, see Table 5.1), and the dashboard talks to it at 115200 baud.

4.2 Control loop timing

`loop()` polls the serial port on every pass and runs the control step only when at least `SAMPLE_TIME_MS = 50` milliseconds have elapsed since the last step. The elapsed time is measured and used as *dt* in the speed computation and in the PI update, so a late iteration does not corrupt the speed estimate. The value 50 ms *must equal* `SAMPLE_DT` in the Python file and `SPEED_LOOP_MS` in the production firmware, because the gains are tuned for that sampling period and because the identification fits are done on data at that spacing.

Blocking calls

- `Serial.setTimeout(20)` in `setup()` bounds `readStringUntil('\n')` to 20 ms. The Arduino default is 1000 ms, which would stall all four loops if a partial line arrived.
- The telemetry line is printed with `Serial.print` at 115200 baud. At about 60–70 characters per line and 20 lines per second, the output uses about 14 kbit/s of the 115.2 kbit/s link, so the transmit buffer does not block.
- There are no interrupt service routines in this firmware. The encoder counting is done by the ESP32 pulse-counter hardware through the `ESP32Encoder` library, so no ISR safety concerns arise in user code.

Memory

All state is statically allocated (four `SimplePID` objects, four encoder objects and small arrays). The only dynamic allocation is the Arduino `String` used for each received command line; command lines are short (under 20 characters), so heap fragmentation is not a practical concern, though a fixed `char` buffer would remove the issue entirely.

4.3 The `SimplePID` class

Listing 4.1 reproduces the control routine. The behaviour is as described in Chapter 3; the specific points of the implementation are:

1. **Open-loop mode** (`open_loop = true`) bypasses the controller and returns `open_pwm`. Integrator and previous error are cleared on every call so that the transition back to closed loop starts clean. This is the mode used for the autotune steps.
2. **Zero target** returns 0 and clears state.
3. **Gain-set selection** by the sign of the target, with a state reset on a direction change.
4. **Integrator clamp** of $\pm 255/K_i$.
5. **Derivative** computed on the error with a one-sample difference. Because the target changes in steps, the first sample after a step produces a large derivative kick when $K_d \neq 0$ (another reason the autotune leaves it at 0).
6. **Output** equals feedforward + P + I + D, clamped to ± 255 and truncated to an integer.

Listing 4.1: Core of `SimplePID::compute()` (abridged).

```
float error = tgt - current;
integral += error * dt;
if (KI > 0.0f) {
  float lim = 255.0f / KI;
  integral = constrain(integral, -lim, lim);
} else {
  integral = 0.0f;
}
float derivative = (dt > 0.0f) ? (error - prev_error) / dt : 0.0f;
prev_error = error;

float ff = tgt * FFG + (tgt > 0.0f ? FF0 : -FF0);
float output = ff + (KP * error) + (KI * integral) + (KD * derivative);
return (int)constrain(output, -255.0f, 255.0f);
```

4.4 Serial command protocol

Commands are single text lines of the form

```
<cmd><sel><value>\n
```

where `cmd` is one character, `sel` is the selector (`A`, `1`, `2`, `3` or `4`) and `value` is a decimal number parsed with `toFloat()`. Lines of two characters or fewer are ignored. Table 4.1 lists every command.

Table 4.1: Serial commands accepted by `PID_TEST.cpp`. Example: `TA30` sets all wheels to 30 rpm; `P25.5` sets K_p of wheel 2 (FR) to 5.5 — note that the value starts right after the selector.

Command	Set	Effect
`T`	target	Speed target in rpm, clamped to ±120; selects closed loop (clears open-loop mode)
`P`, `I`, `D`	forward	K_p, K_i, K_d used when the target is positive
`F`	forward	Feedforward gain, PWM per rpm
`B`	forward	Feedforward offset, PWM counts
`p`, `i`, `d`	reverse	K_p, K_i, K_d used when the target is negative
`f`, `b`	reverse	Feedforward gain and offset for reverse
`M`	open loop	Raw PWM, clamped to ±255, bypassing the controller; negative means reverse. Any later `T` returns to closed loop

Warning. The command letters are case sensitive. In particular `P` sets the *forward* proportional gain and `p` the *reverse* one. The selector uses the digit characters `1`–`4`, not indices 0–3.

4.5 Telemetry

Each control period the firmware prints one line:

```
Target:<rpm> FL:<rpm> FR:<rpm> RL:<rpm> RR:<rpm>
```

The dashboard accepts a line only if it splits on whitespace into exactly five fields and the first contains `Target`; any other line (for example the banner printed at start-up) is ignored. `Target` is the *most recently commanded* target, a single number for display, and not a per-wheel value. Speeds are in rpm after multiplication by the motor direction constant.

4.6 Initial values

All four controllers start with $K_p = 2.0$, $K_i = 5.0$, $K_d = 0$, $F = 3.4$ PWM/rpm, $B = 0$, for both directions. These are the dashboard's "default" values (`DEFAULT_GAINS`) and are what *RESET TO DEFAULT* restores.

Chapter 5

Host Software: `motor_calibrate.py`

5.1 Dependencies and configuration

The dashboard uses `pyserial`, `matplotlib`, `tkinter`, `python-dotenv` and the local module `autotune_math`. All configuration comes from environment variables, optionally set in a `.env` file next to the script. Table 5.1 lists them.

Table 5.1: Environment variables read by the dashboard.

Variable	Default	Meaning
`SERIAL_PORT`	`/dev/ttyUSB0`	Serial port of th `PID_TEST`
`BAUD_RATE`	115200	Must match `Serial` firmware
`GAINS_FILE`	`tuned_gains.json`	Where gains are sav
`ROBOT_PROJECT`	(empty)	PlatformIO project firmware; needed fo and upload
`PIO_CMD`	auto-detected	Path of the `pio` exec
`UPLOAD_PORT`	(empty)	Port used for uploa `SERIAL_PORT`
`PIO_ENV_ROBOT`	`robot`	`platformio.ini` en robot firmware
`PIO_ENV_TEST`	`pid_test`	`platformio.ini` en test firmware
`GAINS_HEADER`	`<ROBOT_PROJECT>/include/gains.h`, or `gains.h` if `ROBOT_PROJECT` is empty	C++ header writte saved; empty string
`AUTOLOAD_GAINS`	1	If 1, load `GAINS_FI` start from defaults

5.2 Tunable constants in the source

Table 5.2 lists the constants at the top of the file that shape the autotune. They are plain Python constants; change them in the source and restart.

Table 5.2: Autotune constants in `motor_calibrate.py`.

Constant	Default	Role
`SAMPLE_DT`	0.05 s	Sample period; must equal `SAMPLE_TIME_MS` in firmware
`TEST_PWMS`	150, 180, 210	Initial open-loop levels; all must clearly move the motor
`TEST_DURATION`	2.5 s	Length of each step; should span several time constants
`SETTLE_STOP`	1.5 s	Coast time between steps
`TAU_C_FACTOR`	0.25	$\tau_c \geq$ this $\times\ \tau$
`TAU_C_L_MULT`	1.25	$\tau_c \geq$ this $\times\ \theta$ (1.0 is the SIMC limit)
`FF_SCALE`	0.0	Starting feedforward, as a fraction of $1/K$
`FF_SPEEDUP`	0.4, 0.7, 1.0	Feedforward fractions tried after PI passes
`ADAPTIVE_LEVELS`	True	Add low-speed identification steps
`REFINE_RPMS`	15, 35	Speeds the extra steps aim for
`MIN_TEST_PWM`	40	Never test below this PWM
`SWEEP_SPEEDS`	10 ... 120 rpm	Speeds of the sweep
`SWEEP_SPREAD_RPM`	2.5 rpm	Tolerance for the sweep
`SWEEP_AFTER_AUTOTUNE`	True	Run the sweep automatically
`VERIFY_RPM`	30 rpm	Verification step (capped at 80 % of fast open-loop speed)
`RIPPLE_MAX_RPM`	3.0 rpm	Peak-to-peak limit
`MAX_OVERSHOOT_PCT`	5 %	Overshoot limit
`DETUNE_FACTOR`	0.75	Multiplier on K_p, K_i per detune round
`MAX_DETUNE_ROUNDS`	3	Maximum detune rounds
`AUTOTUNE_REVERSE`	True	Also tune the reverse direction

5.3 Program structure

The program has four cooperating parts:

1. **Main (GUI) thread.** Tk event loop and Matplotlib canvas. A timer calls `update_plot()` every 100 ms to redraw and to refresh the status line.
2. **Serial reader thread** (`read_serial`). Reads lines, parses the five fields, appends them to five 100-sample ring buffers (`collections.deque`) for the plot, and, when a capture is active, also appends the sample dictionary to the `capture` list.
3. **Worker threads.** Autotune, step test, speed sweep, flashing and reconnect each run in their own daemon thread so the GUI stays responsive.
4. **Shared state.** A set of module-level globals: `autotuning`, `abort_autotune`, `refresh_needed`, `status_text`, `capture`, `flashing`, `updating_gui`, and the per-motor dictionary `motor_params`.

5.3.1 Per-motor memory

`motor_params` stores, for every wheel (keys `"1"`–`"4"`), the ten gains `P,I,D,F,B` and `Pr,Ir,Dr,Fr,Br` plus the last target `T`. The entry `"A"` stores only a target, so the *All* selection can broadcast a speed but can never overwrite individual tuned gains.

5.3.2 Preventing feedback loops in the GUI

Moving a slider by software triggers its callback, which would send a command to the robot. The flag `updating_gui` suppresses sending while the program is only moving sliders to show

stored values; `sync_sliders()` flushes pending Tk idle callbacks while the flag is still set so that no stray command is sent afterwards. While any worker is running, `autotuning` is True and `send_command` refuses to send anything, so manual slider input cannot interfere with a test.

5.3.3 Capture mechanism

A step test records data by assigning a list to the global `capture`, sending the command, sleeping for `TEST_DURATION`, and then swapping `capture` back to `None` and taking the recorded list. A test is rejected if it collected fewer than 80 % of the expected T_{duration}/T_s samples: this protects the fit from serial dropouts and from a mismatch between `SAMPLE_DT` and the firmware.

5.4 Graphical interface

Figure 5.1 shows the layout (to be replaced by a screenshot). The left column holds the controls, the right the live plot of target and four wheel speeds (fixed vertical range ±150 rpm).

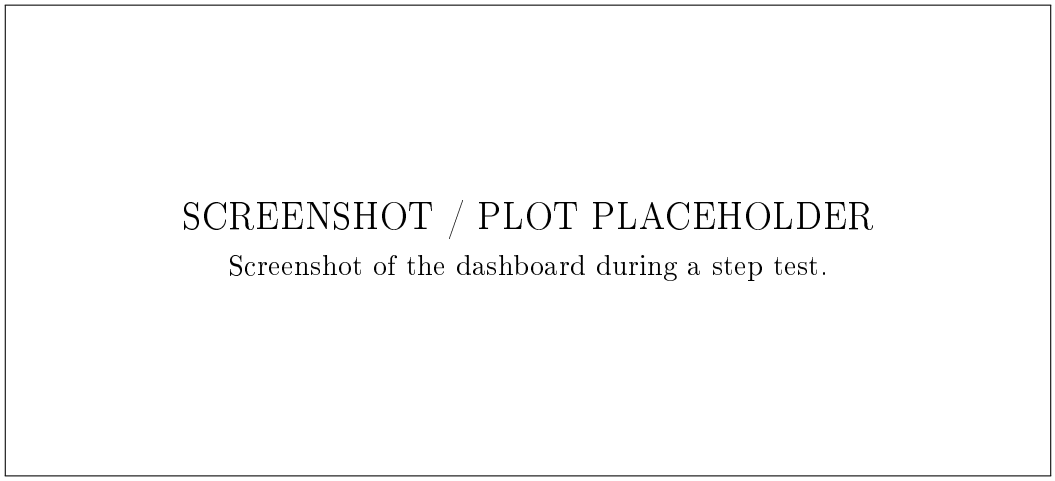


Figure 5.1: Screenshot of the dashboard during a step test.

Table 5.3 explains every control.

Table 5.3: Dashboard controls.

Control	Function
Motor radio buttons	Choose ALL (target only), FL, FR, RL or RR; the sliders then show and edit that wheel
Forward / Reverse gains	Choose which gain set the K_p, K_i, K_d sliders edit
Target RPM slider and box	Send a closed-loop target (± 120 rpm); the box accepts a value and Enter
K_p, K_i, K_d sliders	Edit the selected wheel and direction; disabled when ALL is selected
PAUSE PLOT	Freezes the plot (data keep flowing)
STOP MOTORS	Aborts any test, zeroes all targets, sends `TA0`
AUTOTUNE	Full procedure for the selected wheel or for all
SAVE GAINS	Writes JSON and the C++ header
IMPORT GAINS	Loads a JSON file and applies it
RESET TO DEFAULT	Restores `DEFAULT_GAINS` on all wheels (file on disk untouched)
STEP TEST	Closed-loop ± 30 rpm steps on all wheels with the current gains; prints metrics
SPEED SWEEP	The seven-speed sweep, forward and reverse
UPLOAD PID TEST	Builds and flashes the test firmware, then reconnects
SAVE + UPLOAD ROBOT FIRMWARE	Saves gains, then builds and flashes the robot firmware
RECONNECT	Reopens the serial port after the firmware was changed elsewhere

Chapter 6

Numerical Routines: `autotune_math.py`

6.1 Role of the module

`autotune_math.py` is a pure-Python module. It imports only the standard `math` library and touches no hardware, so each routine can be run and tested offline (Section 6.8 does exactly that). The dashboard calls the four public functions of Table 6.1. The module docstring fixes the plant model of Equation (3.1), $\omega = K(u - u_0)$ with a first-order lag τ and a dead time L, and the firmware feedforward rule $u = rF + \mathrm{sgn}(r)\, u_0$ with $F = \texttt{FF_SCALE}/K$.

Table 6.1: Public functions of `autotune_math.py`.

Function	Called from	Purpose
`identify(levels, rpms, dt)`	`identify_motor`	Dead-band line (K, u_0), then τ and L (Sections 6.3, 6.4)
`common_tau_c(plants, f, m)`	`autotune_worker`	One shared τ_c (Section 6.5)
`simc_pi(K, tau, L, tau_c=)`	`autotune_worker`	K_p, K_i (Section 6.6)
`step_metrics(rpm, target, dt)`	`_verify_step`, `step_test_worker`, `speed_sweep`	Overshoot, rise, settling, ripple, final value (Section 6.7)

6.2 Helper functions

Smoothing

`_smooth(x, n=3)` is a centred moving average whose window shrinks at both ends:

$$\tilde{y}_i = \frac{1}{|W_i|} \sum_{j \in W_i} y_j, \qquad W_i = \{\max(0, i-1), \ldots, \min(N-1, i+1)\}. \tag{6.1}$$

It introduces no net delay but spreads a step edge over three samples (150 ms) and divides an isolated one-sample glitch by three: a one-count error of 1.6 rpm becomes 0.53 rpm. It is non-causal and is used only on recorded data.

Steady value

`_steady(x, frac=0.3)` is the mean of the last $M = \max\{3, \lfloor 0.3N \rfloor\}$ samples. For a 2.5 s record ($N = 50$) that is the last 15 samples (0.75 s). It assumes the response has settled by then; see Section 11.1.

Level crossing

`_crossing(x, level, dt)` returns the first time the series reaches `level`, linearly interpolated. Sample i is placed at $t_i = (i + 1)\,dt$ and the series is taken to start at $(0, 0)$:

$$t^* = t_{i-1} + \frac{\ell - y_{i-1}}{y_i - y_{i-1}}\,(t_i - t_{i-1}). \tag{6.2}$$

If the level is never reached it raises `ValueError`, which `step_metrics` turns into "rise = never".

6.3 Gain and dead-band by regression

`identify` first takes the steady value s_k of every open-loop step k and keeps the points (p_k, s_k) with $s_k \geq 3$ rpm, where p_k is the PWM level. Fewer than two points raise "motor barely moves: raise TEST_PWMS". An ordinary least-squares line through the kept points gives

$$K = \frac{\sum_k (p_k - \bar{p})(s_k - \bar{s})}{\sum_k (p_k - \bar{p})^2}, \qquad u_0 = \bar{p} - \frac{\bar{s}}{K}, \qquad e_{\text{lin}} = \max_k \bigl| s_k - K(p_k - u_0) \bigr|. \tag{6.3}$$

u_0 is the PWM-axis intercept of the line, an extrapolation to zero speed. If the true curve bends near the dead-band (stiction, driver non-linearity) the extrapolation is biased, which is why the refine stage adds levels that produce 15 and 35 rpm. A non-positive K raises "check encoder polarity / PWM levels". The regression is unweighted.

6.4 Time constant and dead time by grid search

For each step with a steady speed of at least 10 rpm (or, if no step reaches 10 rpm, for the fastest step), `_fit_fopdt` fits the model of Equation (3.2) with the steady value held fixed at the measured $\bar{y}$ and only τ and L free:

$$\hat{y}(t_j; \tau, L) = \begin{cases} 0 & t_j \leq L \\ \bar{y}\left[1 - e^{-(t_j - L)/\tau}\right] & t_j > L \end{cases}, \qquad (\hat{\tau}, \hat{L}) = \arg\min_{(\tau, L) \in \mathcal{G}} \sum_j \bigl(y_j - \hat{y}(t_j)\bigr)^2. \tag{6.4}$$

The search grid $\mathcal{G}$ is the product of 75 time constants $\tau_i = 0.03 \cdot 1.06^{\,i}$ (0.03 to 2.24 s, 6 % steps) and 33 delays $L_k = 0.0125\,k$ (0 to 0.4 s, a quarter of the sample period). That is 2475 candidate pairs, about 124 000 exponentials per fit, a fraction of a second in plain Python. The function returns $\hat{\tau}$, $\max(\hat{L}, dt)$ and the rms residual $\sqrt{\text{SSE}/N}$.

`identify` then takes the median of the per-step τ values and of the per-step L values separately (element `len//2` of the sorted list, the upper median when the count is even) and reports the worst rms residual. Taking the median makes the result robust to one bad step.

What L means. The speed sample at t_j is the encoder change over the preceding 50 ms, i.e. the mean speed over that interval, which lags the true speed by about $dt/2 = 25$ ms. The identified L is therefore the motor's own delay plus about 25 ms, and it cannot fall below $dt = 50$ ms because of the floor. In the simulated example of Section 6.8, a true delay of 60 ms was identified as 87.5 ms.

6.5 The shared closed-loop time constant

$$\tau_c = \max_m \ \max\{\ \max(l_{\text{mult}}, 1)\, L_m, \quad f\,\tau_m\ \}, \tag{6.5}$$

over every wheel and direction m, with $f = \texttt{TAU_C_FACTOR} = 0.25$ and $l_{\text{mult}} = \texttt{TAU_C_L_MULT} = 1.25$ in the dashboard. The most delayed or slowest plant therefore sets τ_c for all eight loops, so the four wheels respond alike. The module defaults ($f = 0.5$, $l_{\text{mult}} = 2.0$) are older, more cautious values that the dashboard overrides.

6.6 SIMC gains

With a shared value, `simc_pi` uses $\tau_c \leftarrow \max(\tau_c, L)$ and returns

$$K_p = \min\Big\{\frac{\tau}{K(\tau_c + L)},\ 20\Big\}, \qquad K_i = \min\Big\{\frac{\tau/\,[K(\tau_c + L)]}{\min\{\tau,\ 4(\tau_c + L)\}},\ 60\Big\}, \tag{6.6}$$

where the numerator of the second fraction is the uncapped proportional gain. The caps equal the ranges of the dashboard sliders (0 to 20 and 0 to 60). The product $K_p K = \tau/(\tau_c + L)$ is the same for all plants with the same τ and L, so a stronger motor simply gets a proportionally smaller K_p. If `tau_c` is not passed, the function falls back to $\max(2L,\ f\tau)$; the dashboard always passes it.

6.7 Step metrics

`step_metrics(rpm, target, dt)` evaluates a closed-loop step from rest to a positive target. With $\tilde{y}$ the 3-point-smoothed series and y the raw one:

Output	Definition
`overshoot_pct`	$\max\{0,\ (\max \tilde{y} - r)/r \times 100\}$
`rise90_s`	first time $\tilde{y}$ reaches $0.9\,r$ (Section 6.3's crossing rule), or `None`
`settle_s`	$dt \times (1 + \text{index of the last sample with } \lvert\tilde{y} - r\rvert > \text{band})$, or 0; band $= \max(0.05\,r,\ 1.3\,\text{rpm})$
`final_rpm`	mean of the raw y over the last $\max\{3, \lfloor 0.3N \rfloor\}$ samples
`ripple_pp`	max − min of the raw y over the same samples

The reverse tests multiply the recorded speeds by -1 before calling the function, so the target is always positive.

6.8 Worked example on a simulated motor

To show the routines working end to end, the unmodified `autotune_math.py` was run on a *simulated* motor. This is an illustration with an assumed plant, not a measurement of the robot.

The plant has $K = 0.60$ rpm/PWM (so $K(250-45) \approx 123$ rpm at the top PWM, consistent with the 120 rpm limit), $u_0 = 45$, $\tau = 0.18$ s and a 60 ms delay. It is observed through a 748 count/rev encoder every 50 ms, so the speed is quantised in steps of 1.60 rpm. The simulation repeats the sequence of `identify_motor` (levels 150, 180, 210, then refine levels for 15 and 35 rpm) and of `verify_motor` (PI rounds, then the feedforward speed-up). Replace these tables with your own console output (Table 9.1 onward).

6.8.1 Identification

Table 6.2: True and identified parameters of the simulated motor.

Quantity	True	Identified
K [rpm/PWM]	0.600	0.601
u_0 [PWM]	45.0	45.1
τ [s]	0.180	0.172
L [s]	0.060 (+0.025 measurement lag)	0.0875
Fit rms [rpm]	–	0.82
Linearity error [rpm]	–	0.037
PWM levels used	–	70, 103, 150, 180, 210

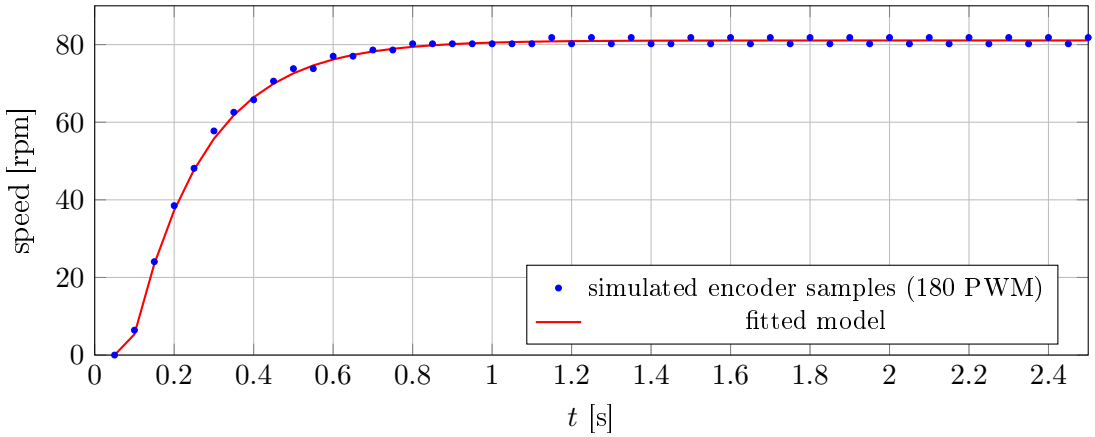


Figure 6.1: Open-loop step at 180 PWM on the simulated motor and the fitted model ($\hat{\tau} = 0.183$ s, $\hat{L} = 0.0875$ s). The steps in the data are single encoder counts (1.6 rpm). Simulation, not measured data.

6.8.2 Gains and verification

The shared τ_c is $\max(1.25 \times 0.0875,\ 0.25 \times 0.172) = 0.109$ s. `simc_pi` then gives $K_p = 1.46$ and $K_i = 8.46$, with feedforward offset $B = 45.1$ and $F = 0$ (`FF_SCALE` $= 0$).

Table 6.3: `verify_motor` on the simulated motor, 30 rpm step. A round passes if overshoot $\leq 5\,\%$ and ripple ≤ 3 rpm.

Round	K_p	K_i	Overshoot [%]	Rise 90 % [s]	Settle [s]	Ripple [rpm]	Result
1	1.46	8.46	7.0	0.28	0.50	1.6	fail, detune
2	1.09	6.35	1.6	0.39	0.45	1.6	pass
FF 0.4/K	1.09	6.35	12.3	0.27	0.70	1.6	fail, stop

The first round overshoots by 7 %, so both gains are multiplied by 0.75 and the second round passes. The feedforward search then tries $F = 0.4/K = 0.666$ and fails on overshoot (12.3 %), which ends the search, so the final feedforward stays at zero.

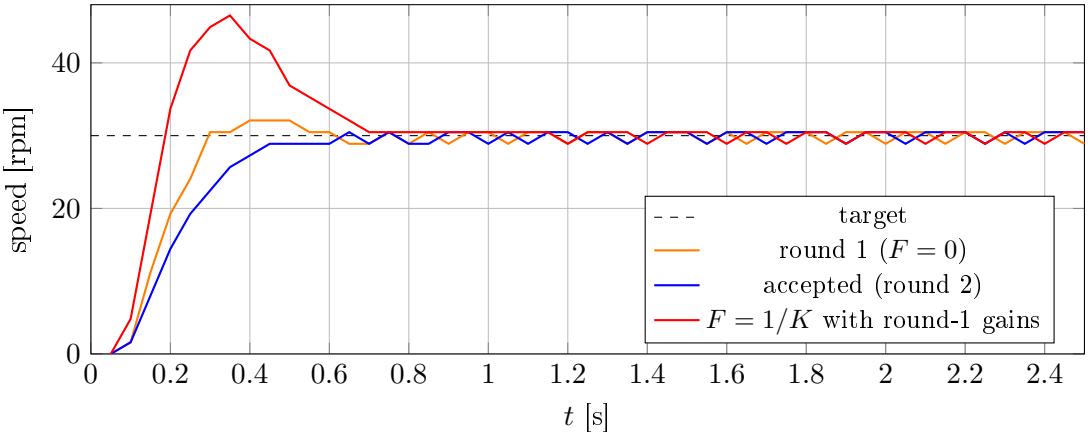


Figure 6.2: Closed-loop 30 rpm step on the simulated motor. Round 1 overshoots, the detuned round 2 is accepted, and adding feedforward to a plain PI loop makes the overshoot worse. Simulation, not measured data.

6.8.3 Effect of τ_c and of feedforward

Table 6.4: Effect of the design parameter τ_c (simulated motor, $F = 0$, 30 rpm step). Values of τ_c below L give the same gains, because of the $\max(\tau_c, L)$ floor.

τ_c [s]	K_p	K_i	Overshoot [%]	Rise 90 % [s]	Settle [s]
0.06	1.64	9.51	10.5	0.25	0.50
0.08	1.64	9.51	10.5	0.25	0.50
0.12	1.38	8.02	3.4	0.30	0.30
0.18	1.07	6.22	1.6	0.41	0.45
0.30	0.74	4.30	1.6	0.76	0.85
0.50	0.49	2.83	0.0	1.22	1.60

Table 6.5: Effect of the feedforward fraction f ($F = f/K$) with the round-1 gains and no detuning (simulated motor).

f	F [PWM/rpm]	Overshoot [%]	Rise 90 % [s]	Settle [s]	Ripple [rpm]
0.0	0.000	7.0	0.28	0.50	1.6
0.4	0.666	23.0	0.22	0.55	1.6
0.7	1.165	37.3	0.20	0.60	1.6
1.0	1.665	49.7	0.18	0.65	1.6

Three things follow, and all are properties of this simulated plant. (i) Identification recovers K and u_0 to better than 0.5 % and τ to about 5 %. (ii) With $l_{\text{mult}} = 1.25$ the autotune starts at the edge of acceptable overshoot and typically needs one detune round, while τ_c of about 1.5 to 2 times L would start closer to the final answer (the dashboard comment notes that 2.0 was the earlier setting). (iii) A plain PI loop with feedforward overshoots more as f grows, because the integrator accumulates error during the rise even though feedforward already supplies the steady-state drive. The speed-up search therefore stops at its first step here. Feedforward only

pays off with a setpoint ramp or filter, or with setpoint weighting on the proportional term; whether your motors behave the same way is visible in the console lines "FF 0.4/K".

Chapter 7

The Autotune Algorithm Step by Step

7.1 Overview

The autotune is one function, `autotune_worker(selection)`, running in a thread. Figure 7.1 summarises it.

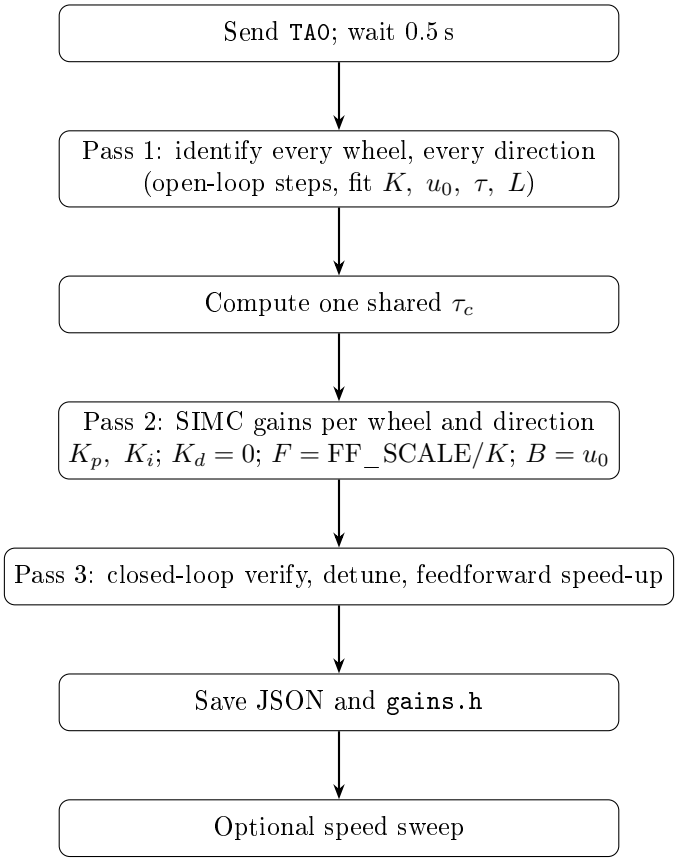


Figure 7.1: Flow of `autotune_worker`. Any step can be aborted with STOP MOTORS.

7.2 Pass 1: identification

For each wheel and direction `identify_motor(m, d)` performs:

1. **Stage 1.** For each PWM in `TEST_PWMS`: send `M<m><pwm>` (reverse uses −pwm), record 2.5 s of speed, send `T<m>0`, then `wait_stopped`, which sleeps 1.5 s and waits a further 1.5 s if the last measured speed is still above 2 rpm.
2. Fit the model to the three traces (reverse data are sign-flipped so the fit always works on positive magnitudes).
3. **Stage 2** (if `ADAPTIVE_LEVELS`): for each speed in `REFINE_RPMS` compute the PWM from Equation (3.3), skip it if within 8 counts of an existing level, otherwise run the step. Refit with all levels.
4. Print the result line.

If at any point the user presses STOP MOTORS, the function returns `None` and the whole autotune reports "aborted".

The three initial levels are all high (150–210), which is a deliberate choice: the motors must clearly move so that the step shows a clean rise. The disadvantage is that 150–210 counts without load produce speeds well above the range the robot will be driven in, hence the refine stage.

7.3 Pass 2: computing gains

With all plants identified, a single τ_c is computed over all of them (Equation (3.7)) and printed. Then for each wheel and direction the SIMC gains are computed and stored:

$$K_p,\ K_i \leftarrow \texttt{simc_pi}(K, \tau, \theta, \tau_c), \quad K_d \leftarrow 0, \quad F \leftarrow \texttt{FF_SCALE}/K, \quad B \leftarrow u_0.$$

K_p and K_i are rounded to two decimals, F to three and B to one. If `AUTOTUNE_REVERSE` is off, the reverse set is copied from the forward set.

7.4 Pass 3: verification

For each (wheel, direction) the verification of Section 3.6 is run as described in Chapter 3: PI detune loop first, then the feedforward speed-up. Each test pushes all ten gains to the wheel with `push_gains` (10 ms between lines to stay below the ESP32 receive buffer), runs the step, waits for the wheel to stop, computes the metrics and prints them. The final feedforward is pushed again at the end.

7.5 Saving

On success the dashboard calls `save_gains()`, which (i) copies any existing JSON to `.bak`, (ii) writes the new JSON, and (iii) writes the header `gains.h`. The JSON structure is

```
{ "1": {"P":..,"I":..,"D":..,"F":..,"B":..,"Pr":..,"Ir":..,"Dr":..,"Fr":..,"Br":..},
  "2": {...}, "3": {...}, "4": {...} }
```

and the header is

```
struct Gain { float kp, ki, kd, ff, off; };
struct MotorGains { Gain fwd, rev; };
constexpr MotorGains GAINS_FL = {{kp, ki, kd, ff, off}, {kp, ki, kd, ff, off}};
// ... GAINS_FR, GAINS_RL, GAINS_RR
```

The production firmware includes this header and builds its per-wheel controllers from it. Importing an older JSON file that lacks the reverse set is supported: the reverse values are filled from the forward ones.

7.6 Estimated duration

A rough estimate from the constants (not measured): each open-loop step takes 2.5 s plus 1.5–3 s of settling, five steps per wheel and direction give about 20–25 s, and the closed-loop verifications take 4–7 steps of similar length. With four wheels and both directions the identification takes roughly 3 minutes and the verification up to about 4 minutes, plus about 1 minute for the sweep. Allow 5–10 minutes in total. **[TODO: Replace with the measured duration.]**

Chapter 8

Other Functions of the Dashboard

8.1 Step test

`step_test_worker` applies a closed-loop step from 0 to +30 and then to −30 rpm on *all* wheels simultaneously with the *current* gains and prints the five metrics per wheel and direction together with the gains in use. Its intended use is a before/after comparison: run it after RESET TO DEFAULT, then after IMPORT or AUTOTUNE, and compare overshoot, rise time and ripple.

8.2 Speed sweep

See Section 3.7. It runs automatically after a successful autotune and can also be started on demand. The summary status line reports the number of flagged speeds; per-speed details are printed to the console, with the marker `<- CHECK` on flagged lines.

8.3 Flashing from the dashboard

Two buttons run `pio run -e <env> -t upload -d <ROBOT_PROJECT> -upload-port <port>`:

- **UPLOAD PID TEST** flashes the test environment and then reconnects the dashboard, pushing the current gains.
- **SAVE + UPLOAD ROBOT FIRMWARE** saves the gains (JSON and header) first and then flashes the robot environment. The dashboard serial port is closed during this, and afterwards the dashboard remains disconnected: use UPLOAD PID TEST to tune again.

Output of `pio` is streamed to the console and to the status line. If `ROBOT_PROJECT` is not set the dashboard says so instead of failing silently.

Chapter 9

Operating Procedure

9.1 One-time set-up

1. Install the Python packages:

   ```
   pip install pyserial matplotlib python-dotenv
   ```

   (`tkinter` comes with most Python installs; on Debian/Ubuntu use `sudo apt install python3-tk`.)

2. Create `.env` next to `motor_calibrate.py`, for example:

   ```
   SERIAL_PORT=/dev/ttyUSB0
   ROBOT_PROJECT=/home/you/ROBOCAR_ROS
   PIO_ENV_ROBOT=robot
   PIO_ENV_TEST=pid_test
   ```

3. Make sure the two environments exist in `platformio.ini`, and that the `pid_test` environment compiles `PID_TEST.cpp` and not `main.cpp`. **[TODO: Insert the relevant `platformio.ini` excerpt.]**

4. Check in `pins.h` that the pin numbers and `MOTOR_DIR_*` match the wiring (Section 9.2).

5. On Linux, add your user to the `dialout` group to access the serial port.

9.2 Direction check (before any tuning)

For each wheel in turn, with the wheel lifted, send a small positive target (for example `T110` for FL at 10 rpm) and verify that (i) the wheel turns in the direction the robot would roll *forward* and (ii) the plotted speed is *positive*. If the direction is wrong, flip the sign of `MOTOR_DIR_*` for that wheel in `pins.h`. Because that constant multiplies both the PWM and the encoder reading, flipping it corrects both the direction and the feedback sign at once. Do not proceed if any wheel plots a negative speed for a forward command, since a positive-feedback loop will run away.

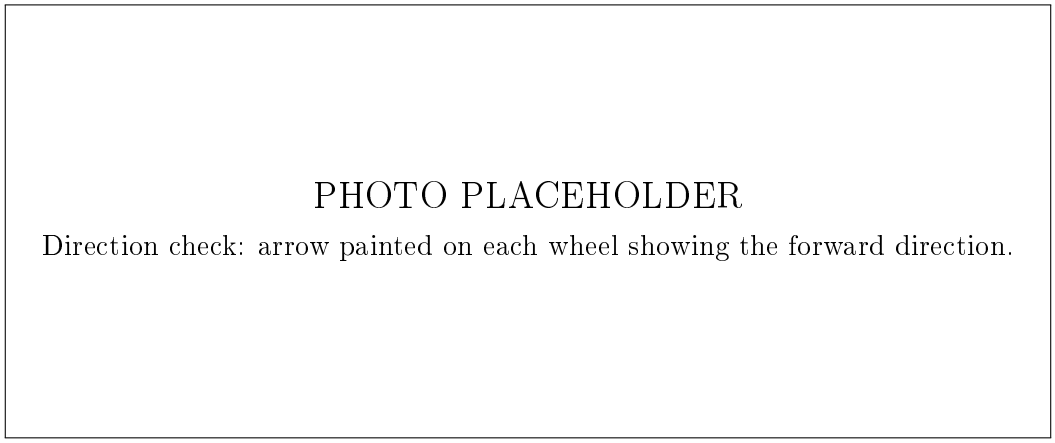


Figure 9.1: Direction check: arrow painted on each wheel showing the forward direction.

9.3 Tuning session

1. Charge the battery. A weak battery changes K and the dead-band between runs and has a large effect on the result. **[TODO: Record the battery voltage at the start.]**
2. Lift the robot, all wheels free (Section 1.4).
3. Flash the test firmware with UPLOAD PID TEST or from PlatformIO. Start the dashboard: `python3 motor_calibrate.py`.
4. Confirm that the plot shows the five traces at zero. If the port does not open, the program prints the error and exits.
5. Select ALL and press AUTOTUNE. Watch the console, not only the status line.
6. When the status line says "Autotune done, gains saved", read the printed identification lines (Table 9.1) and the sweep lines.
7. Press STEP TEST and compare against a step test taken after RESET TO DEFAULT. **[TODO: Insert the two console outputs.]**
8. Press SAVE + UPLOAD ROBOT FIRMWARE to put the tuned gains into the robot, or build from PlatformIO. The header `gains.h` was already written when gains were saved.
9. Put the robot on the floor and repeat a step test on the ground (Section 9.5).

SCREENSHOT / PLOT PLACEHOLDER

Plot of the four wheel speeds during STEP TEST after autotune.

Figure 9.2: Plot of the four wheel speeds during STEP TEST after autotune.

9.4 Recording the results

Table 9.1: Identified plant parameters (copy from the console). **[TODO: Fill in.]**

Wheel	Dir	K [rpm/PWM]	u_0 [PWM]	τ [s]	L [s]	fit rms [rpm]	lin. err [rpm]
FL	fwd						
FL	rev						
FR	fwd						
FR	rev						
RL	fwd						
RL	rev						
RR	fwd						
RR	rev						

Table 9.2: Final gains. **[TODO: Fill in from `tuned_gains.json`.]**

Wheel	Dir	K_p	K_i	K_d	F	B
FL	fwd			0		
FL	rev			0		
FR	fwd			0		
FR	rev			0		
RL	fwd			0		
RL	rev			0		
RR	fwd			0		
RR	rev			0		

Table 9.3: Closed-loop verification at 30 rpm. **[TODO: Fill in.]**

Wheel	Dir	Overshoot [%]	Rise 90% [s]	Settle [s]	Ripple [rpm pp]	Final [rpm]
FL	fwd					
FL	rev					
FR	fwd					
FR	rev					
RL	fwd					
RL	rev					
RR	fwd					
RR	rev					

9.5 Tuning on the floor versus on the stand

The autotune is performed with the wheels in the air. The motors then drive only their own rotor, gearbox and wheel inertia. On the floor the robot's mass, rolling resistance and, for a skid-steer, the lateral scrub when turning add load. The consequences are a lower effective K, a larger dead-band, and a longer τ. The gains obtained on the stand are therefore somewhat more aggressive than optimal for the loaded robot, and they should always be confirmed on the floor with a step test. If the floor behaviour is noticeably worse (overshoot, oscillation), either repeat the autotune with the robot on the floor, at the cost of the robot moving by itself during the test, or make the tuning more conservative by raising `TAU_C_FACTOR` and `TAU_C_L_MULT` (a larger τ_c gives lower gains).

9.6 When to retune

Repeat the procedure after changing a motor, gearbox, wheel or tyre, after changing the battery type, after the PWM frequency is altered, after changing `TICKS_PER_REV` or `SAMPLE_TIME_MS`, or whenever the step test shows visibly slower or less damped behaviour than at commissioning.

Chapter 10

Troubleshooting

Symptom	Likely cause	What to check or do
Script exits with "Error opening serial port"	Wrong port, port in use by another program, or permissions	Close the Serial Monitor; check `SERIAL_PORT`; check `dialout` group
Plot stays flat at zero	Wrong baud rate or wrong firmware flashed	`BAUD_RATE` must be 115200; confirm `pid_test` was flashed, not the robot firmware (which uses 460800 baud and micro-ROS)
Autotune stops with "only N samples in 2.5 s"	Serial data lost, or `SAMPLE_DT` differs from firmware	Check cable; check `SAMPLE_TIME_MS = 50`; avoid running other heavy programs
Wheels do not move during open-loop steps	Motor power off or battery empty, driver not enabled, wiring	Check motor supply voltage and driver enable; the board can run on USB with dead batteries while the wheels stay still
Identification reports a very large L or fit error	Slipping coupling, loose encoder, battery sag during the step	Inspect mechanics; use a fresh battery; check for encoder noise
A wheel plots negative speed for a forward command	`MOTOR_DIR` wrong for that wheel	Section 9.2
Wheel runs away to full speed after autotune	Encoder not counting or counting the wrong way	Check encoder wiring and pull-ups; rerun the direction check
Verification keeps failing and detunes three times	Quantisation ripple above the limit, or genuine oscillation	Look at the ripple value: if it is one or two counts (1.6 or 3.2 rpm), raise `RIPPLE_MAX_RPM` slightly above two counts; otherwise inspect the mechanics
Sweep flags 100 and 120 rpm only	PWM saturation	Compare with Equation (3.8); the motor cannot reach those speeds on this battery
Sweep spread large at low speed	Different dead-bands or stiction	Check u_0 values in Table 9.1; mechanical friction differences
Upload fails with "pio not found"	`PIO_CMD` not set	Typically `~/.platformio/penv/bin/pio`

Symptom	Likely cause	What to check or do
After SAVE + UPLOAD ROBOT the dashboard no longer responds	Expected: the dashboard port is closed and the robot firmware is on the board	Press UPLOAD PID TEST to return to tuning
Robot drifts in a straight line on the floor despite good tuning	Wheel-speed loops do not guarantee path; slip and uneven friction	Heading hold on the robot firmware, see its own documentation

Chapter 11

Known Limitations and Recommendations

11.1 Review of `autotune_math.py`

The module was read in full and run on a simulated motor (Chapter 6), so this report now states what it does rather than what it was assumed to do. The points below are properties worth knowing. None needs an immediate fix.

1. **The identified dead time L includes the measurement lag and cannot go below one sample.** Each speed sample is the mean over the preceding 50 ms, so it lags the true speed by about 25 ms, and `_fit_fopdt` returns $\max(L,\ dt)$. In the simulation a true 60 ms delay was identified as 87.5 ms. That is the right quantity for tuning (the controller sees the same delayed signal), but L is a property of motor plus sensor, and delays shorter than 50 ms are not resolved.

2. **K_i is computed from the uncapped K_p.** In `simc_pi` the caps are applied in the return statement only, so if K_p is capped at 20 the integral gain still corresponds to the larger value, and the effective integral time K_p/K_i is shorter than τ_I. For a very small plant gain ($K = 0.02$ rpm/PWM, $\tau = 0.2$ s, $L = 0.05$ s) the function returns (20, 60) whereas the design integral time is 0.2 s and the returned pair gives 0.33 s. Realistic gains for this robot (a few tenths of rpm per PWM count) are far from the caps, so this is only a note.

3. **The settling band is slightly below one encoder count.** At a 30 rpm target the band is $\max(1.5,\ 1.3) = 1.5$ rpm while one count is 1.6 rpm (the 1.3 floor dates from the earlier 1.2 rpm resolution assumption). The 3-point smoothing reduces an isolated one-count excursion to about 0.5 rpm, which stays inside the band, but sustained one-count hunting at the band edge can still push `settle_s` to the end of the window. Raise the floor to about 2 rpm if settling times look erratic.

4. **`settle_s` saturates at the window length.** A trace that never settles returns $N \cdot dt$ (2.5 s), the same as one that settles exactly at the end; one that is always inside the band returns 0.

5. **The steady value assumes the speed has settled in the last 30 % of the window.** For $N = 50$ that is the last 0.75 s, from 1.75 s on. This is fine for $\tau \lesssim 0.4$ s, but for $\tau = 1$ s the mean is still about 10 % low and K is under-estimated by the same amount.

6. **`lin_err_rpm` is the worst residual, not an rms value,** and with three initial levels the line has one degree of freedom, so a small value says little until the refine levels are added.

7. **Module defaults differ from the dashboard.** `common_tau_c` and `simc_pi` default to factors of 0.5 and 2.0 (the older, more robust setting); the dashboard passes 0.25 and 1.25 for the first and always passes `tau_c` explicitly to the second, so the defaults are never used.

8. **Comments.** The docstring quoted a quantisation of 1.2 rpm; it is 1.6 rpm, and the comment on `_TAUS` gave an upper limit of about 2.4 s where the grid ends at 2.24 s. Both comments are corrected in the supplied copy of the file; no code changed.

11.2 Observations on the supplied source

1. **Ripple limit.** `RIPPLE_MAX_RPM = 3.0` is just under two encoder counts (3.2 rpm), so a steady-state ripple of exactly two counts is flagged. If the verification fails only on ripple of that size, raise the limit to about 3.5 rpm.

2. **Gains tuned unloaded.** See Section 9.5.

3. **Single sample period.** The dashboard, test firmware and robot firmware each define the sample period separately (50 ms). A mismatch silently invalidates the gains. A shared header or a startup handshake would remove the risk.

4. **Header written even on partial failure.** `save_gains` is called after pass 3 regardless of how many verification rounds failed; check the console for "FF"/"PI round" lines that end in a failed state before trusting the file.

5. **Dynamic `String` in the firmware.** Harmless at this command size, but a fixed buffer would be more robust.

6. **No derivative filter.** Do not raise K_d without adding speed filtering.

7. **Constant gains.** The model is linear; a gain schedule over speed could improve the low-speed region, but the sweep indicates whether this is needed.

11.3 Possible extensions

- Optional tuning on the floor with a safety limit on wheel travel.
- Automatic battery-voltage normalisation (scale PWM by $V_{\text{nom}}/V_{\text{bat}}$) so that the gains hold as the battery discharges.
- A short burst of random excitation for a more general identification.
- Automatic generation of this report's tables from the console log.

Appendix A

Quick Reference

A.1 Command cheat sheet

`TA30`	`all wheels to +30 rpm`	`T2-20`	`wheel 2 (FR) to -20 rpm`
`TA0`	`stop all`	`M3150`	`wheel 3 (RL) raw PWM +150, open loop`
`P12.5`	`wheel 1 forward Kp = 2.5`	`p12.5`	`wheel 1 reverse Kp = 2.5`
`F43.4`	`wheel 4 forward ff = 3.4`	`B440`	`wheel 4 forward offset = 40`

(The value follows the selector directly, so `P12.5` is command `P`, selector `1`, value `2.5`.)

A.2 Procedure on one page

1. Charged battery, wheels lifted, direction check passed.
2. Flash `pid_test`; start `motor_calibrate.py`.
3. Select ALL, press AUTOTUNE; wait about 5–10 minutes.
4. Read identification, verification and sweep lines; record the tables.
5. STEP TEST; compare with defaults.
6. SAVE + UPLOAD ROBOT FIRMWARE.
7. Confirm on the floor.

Appendix B

Fill-in Photo Checklist

- Robot on stand (Figure 1.1)
- Motor, encoder and driver wiring (Figure 2.1)
- Wheel forward-direction marks (Figure 9.1)
- Dashboard screenshot (Figure 5.1)
- Step-test plot (Figure 9.2)
- Optional: before/after step-test comparison; speed-sweep console output.